

Clinic Practice — Session Revision Manual

Project: clinic-practice

Author: Kai (learning track toward OpenMRS)

Date: 31 May 2026

Stack: Spring Boot 4.x · Java 17 · Spring Data JPA · MySQL (Docker) · Postman

1. What this project is

A small **REST API** for a fake clinic, separate from `openmrs-dev`. You practice the same backend ideas OpenMRS uses:

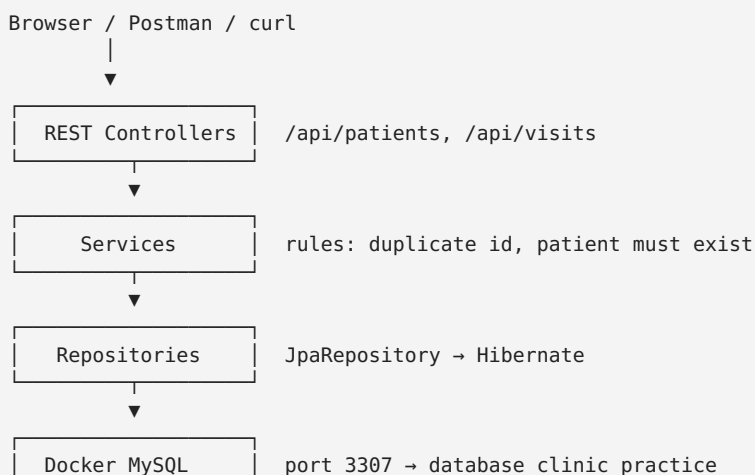
HTTP request → Controller → Service → Repository → Database

Layer	Your classes	Role
Web	PatientController, VisitController	URLs, HTTP status codes, JSON in/out
Service	PatientServiceImpl, VisitServiceImpl	Business rules
Repository	PatientRepository, VisitRepository	Database access (Spring generates SQL)
Model	Patient, Visit	Tables + JSON shape (@Entity)
Config	DataInitializer	Seed data on startup
Errors	GlobalExceptionHandler, ApiError	Central 400 responses

2. Sessions completed

Session	Topic	Key skills
A-B	Spring Boot + HelloController	@SpringBootApplication, ./mvnw spring-boot:run
C	Patients GET	@RestController, @GetMapping, service + in-memory Map
D	POST create	@PostMapping, @RequestBody, HTTP 201
E	PUT + DELETE	@PathVariable, Optional, 204 No Content
F	GlobalExceptionHandler	@ControllerAdvice, one place for 400 errors
G3	Validation	@Valid, @NotBlank, MethodArgumentNotValidException
G2	MySQL + JPA	Docker MySQL, JpaRepository, ddl-auto=update
Visits	Second table	findByPatientId, FK-style patientId, full CRUD

3. Architecture diagram



4. API reference

Base URL: `http://localhost:8080`

Patients

Method	Path	Success	Notes
GET	<code>/api/patients</code>	200	List all
GET	<code>/api/patients/{id}</code>	200 / 404	One patient
POST	<code>/api/patients</code>	201	Body needs <code>id</code> , <code>givenName</code> , <code>familyName</code>
PUT	<code>/api/patients/{id}</code>	200 / 404	Body: names only (id from URL)
DELETE	<code>/api/patients/{id}</code>	204 / 404	No body

Visits

Method	Path	Success	Notes
GET	<code>/api/visits</code>	200	List all
GET	<code>/api/visits/patient/{patientId}</code>	200	Visits for one patient
GET	<code>/api/visits/{id}</code>	200 / 404	One visit
POST	<code>/api/visits</code>	201	<code>patientId</code> must exist in <code>patients</code>
PUT	<code>/api/visits/{id}</code>	200 / 404	Body must include <code>patientId</code>
DELETE	<code>/api/visits/{id}</code>	204 / 404	No body

Hello

Method	Path
GET	<code>/api/hello</code>

5. How to run the project

Start database (Docker)

```
cd clinic-practice
docker compose up -d
docker compose ps    # wait for healthy
```

MySQL is on **host port 3307** (not 3306 — local MariaDB uses 3306).

Start application

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
./mvnw spring-boot:run
```

Wait for: `Started ClinicPracticeApplication`

Test database connection

```
mysql -h 127.0.0.1 -P 3307 -u clinic -pclinic_pass clinic_practice -e "SHOW TABLES;"
```

Stop

```
# App: Ctrl+C
docker compose stop
```

6. Configuration (application.properties)

```
spring.datasource.url=jdbc:mysql://localhost:3307/clinic_practice
spring.datasource.username=clinic
spring.datasource.password=clinic_pass
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Setting	Meaning
3307	Docker MySQL port on your machine
ddl-auto=update	Hibernate creates/updates tables (learning only; OpenMRS uses Liquibase)
show-sql=true	Print SQL in console while learning

7. Important Spring annotations

Annotation	Where	Meaning
@Entity	Model	This class is a database table
@Id	Model field	Primary key
@Table(name = "...")	Model	Table name
@RestController	Controller	REST API + JSON responses
@RequestMapping("/api/...")	Controller	Base path
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	Method	HTTP verb + path
@PathVariable	Method param	Bind {id} from URL — required
@RequestBody	Method param	JSON → Java object
@Valid	Method param	Run validation annotations on body
@Service	Service impl	Spring bean for business logic
@ControllerAdvice	Exception handler	Global error handling
@NotBlank	Model field	Validation: not null/empty

8. JPA repository methods (memorize these)

You might write (wrong)	Correct
getAll()	findAll()
existById()	existsById()
findOne()	findById() → returns Optional
deleteById()	deleteById()

Custom query by naming:

```
List<Visit> findByPatientId(String patientId);
```

9. Bugs you hit and fixes (revision gold)

Typos break compile

- patientRepositroy vs patientRepository — Java names must match exactly
- JpaRepositroy → JpaRepository
- Missing semicolon in constructor

@PathVariable **missing**

URL: `/api/visits/v10` but parameter `String id` without `@PathVariable` → `id` is `null` → `existsById(null)` → **"The given id must not be null"**

Fix: `delete(@PathVariable String id)`

`@PutMapping` **without** `/{id}`

`@PutMapping` alone = only `PUT /api/visits`, not `PUT /api/visits/v10` → **405 Method Not Allowed**

Fix: `@PutMapping("/{id}")`

DELETE logic inverted

```
// WRONG – returns false when visit EXISTS
if (visitRepository.existsById(id)) {
    return false;
}

// CORRECT
if (!visitRepository.existsById(id)) {
    return false;
}
visitRepository.deleteById(id);
return true;
```

PUT body missing `patientId`

Update calls `patientRepository.existsById(visit.getPatientId())`. If body has no `patientId`, same **"The given id must not be null"** error.

Wrong MySQL port

`3397` in properties → **Connection refused**. Docker uses **3307**.

Maven BUILD SUCCESS vs app failed

Maven can print **BUILD SUCCESS** even when the app crashed on startup. Look for `Application run failed` in logs.

10. OpenMRS comparison (G1)

Clinic practice (you)	OpenMRS appointments module
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> (older style)
<code>@GetMapping("/{id}")</code>	<code>@RequestMapping(method = GET, value="/{uuid}")</code>
Constructor injection	Often <code>@Autowired</code> fields
Spring Boot embedded Tomcat	WAR on external Tomcat
<code>/api/patients</code>	<code>/openmrs/ws/rest/v1/appointments</code>
<code>ddl-auto=update</code>	Liquibase XML migrations
Direct <code>Patient</code> entity in API	Often DTO + Mapper (<code>AppointmentRequest</code>)

Same idea: Controller → Service → data. Your app is modern Boot; OpenMRS modules are older style but the layers match.

11. Postman

Import: `postman/clinic-practice.postman_collection.json`

Copy-paste bodies: `postman/POSTMAN-TEST-DATA.md`

Remember: Browsers only do GET easily. Use Postman or `curl` for POST, PUT, DELETE.

12. Seed data (`DataInitializer`)

On empty database:

Type	id	Notes
Patient	1, 2	First / Second Patient
Visit	v1, v2	Both for patient 1

Runs only when `count() == 0` for each table.

13. What to learn next

1. **DTOs** — separate API JSON from `@Entity` (like OpenMRS `AppointmentRequest`)
2. **OpenMRS module** — one REST endpoint in `openmrs-dev` with SDK
3. **@ManyToMany** — JPA relationship Visit → Patient
4. **Liquibase** — how OpenMRS manages schema instead of `ddl-auto`
5. **Spring Security** — auth on REST (OpenMRS uses Basic Auth)

14. Quick commands cheat sheet

```
# Docker DB
docker compose up -d
docker compose ps

# Run app
./mvnw spring-boot:run

# Compile only
./mvnw compile

# MySQL shell
mysql -h 127.0.0.1 -P 3307 -u clinic -pclinic_pass clinic_practice

# Example curl
curl http://localhost:8080/api/patients
curl -X POST http://localhost:8080/api/visits \
  -H "Content-Type: application/json" \
  -d '{"id":"v10","patientId":"1","visitDate":"2026-05-31","reason":"Checkup"}'
```

End of revision manual — clinic-practice learning project.